了解APP架构设计

什么是架构

- 架构设计理念
- 架构设计思维
- 架构设计模式原则
- 架构设计步骤

一、架构设计理念

- 架构设计三要素
- 软件架构设计的关注点

架构设计三要素

- 实用： 避免过度设计，如果设计复杂度较高，应考虑软件维护者能否接受这样的架构设计，以及可能带来的长期维护成本；
- 坚固： 面对外界的变化，相应要迅速及时。同时要灵活可扩展；
- 美观： 代码要简洁，易于阅读，维护，能使维护者从直观上对功能产生更清晰的认识，从而能够快速的处理变化。

软件架构设计的关注点

- 变化： 架构设计更多的是为可能出现的“变化”服务。在较恶劣的“变化”到来之时，维护者们为了快速交付任务，可能会随意堆积代码，代码交付就像马拉松最后的冲刺，临近极限时甚至会像博尔特短跑冲刺，代码肆意堆积，任由病毒滋生，从而产生一个令人难以理解的软件系统，这样的软件系统看起来几乎是不可修改的，每一个新加入的成员都会为其复杂度而感到惊讶。前期的架构设计应该做好“选址”，尽量避免因为“天灾人祸”的降临而出现“大厦将倾”的现象。

二、架构设计思维

- 简化思维
- 分层思维
- 分治思维
- 迭代思维

简化思维

简化指降低事物的复杂度，用更少的细节来代替较多的细节

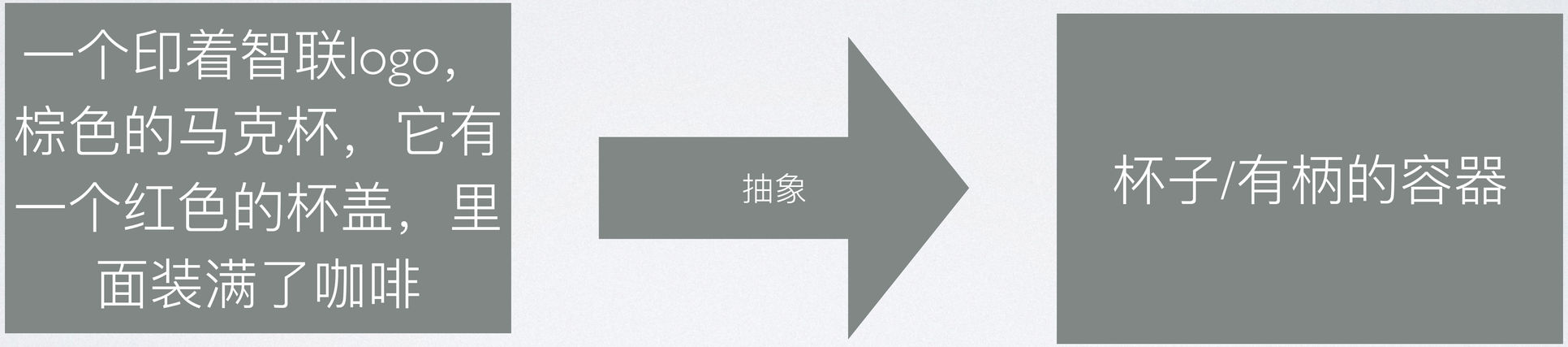
一个印着智联logo，棕色的马克杯，它有一个红色的杯盖，里面装满了咖啡

简化

有红色杯盖的棕色马克杯

在软件架构中，抽象也是一种特殊的简化，相比较简化而言，抽象降低复杂度的力度更大。抽象意味着删除或是隐藏事物的细节，而不会破坏事物本身。

简化 != 抽象



在软件架构设计中，简化系统组件可以避免过度设计，系统组件的抽象程度直接影响了一个系统整体的复杂度。简化或是抽象可以提升软件设计的一致性，使代码更加清晰。

掌握简化和抽象思维，需要开发者对事物有宏观的理解，对组织形态有完整的认识，简化复杂部分，隐藏不必暴露的细节，实现降低复杂度和提升代码可读性的目标。

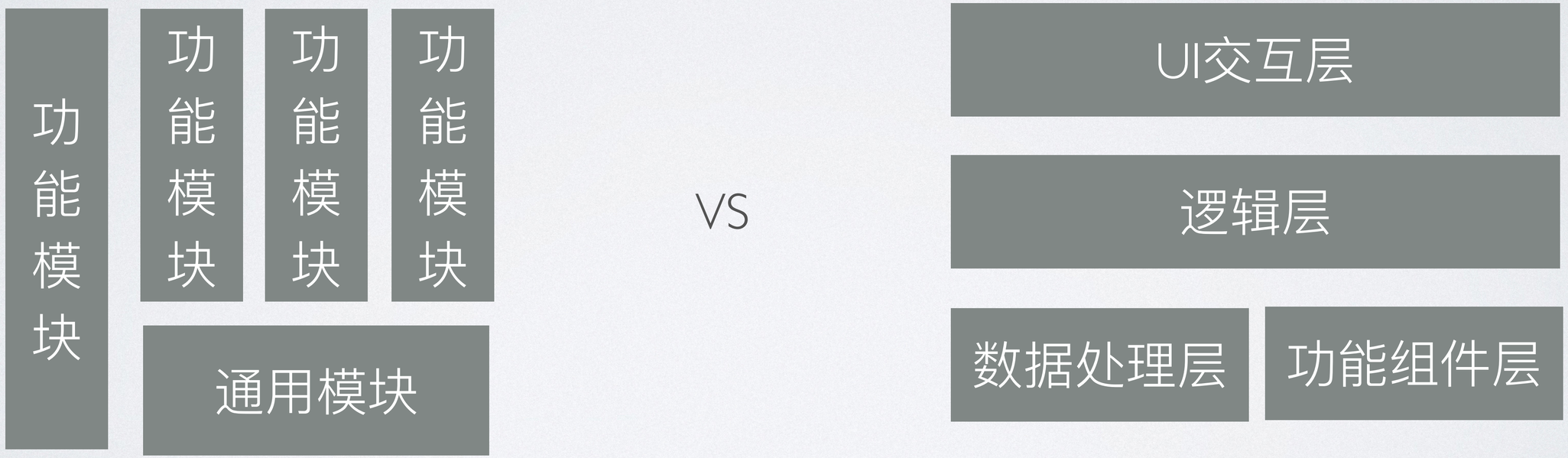
分层思维

几乎每一种架构都会用到分层思维，以达到关注点分离的目标。

比如，View与Model关注点分离，可以提升软件组成部分的可操控能力，这也是MVC的架构的核心设计思想之一。

模块划分

模块划分时，有人习惯划分功能模块，有人则喜欢分层，这两种设计思想分别为关注垂直维度切分和关注水平维度切分，并不矛盾



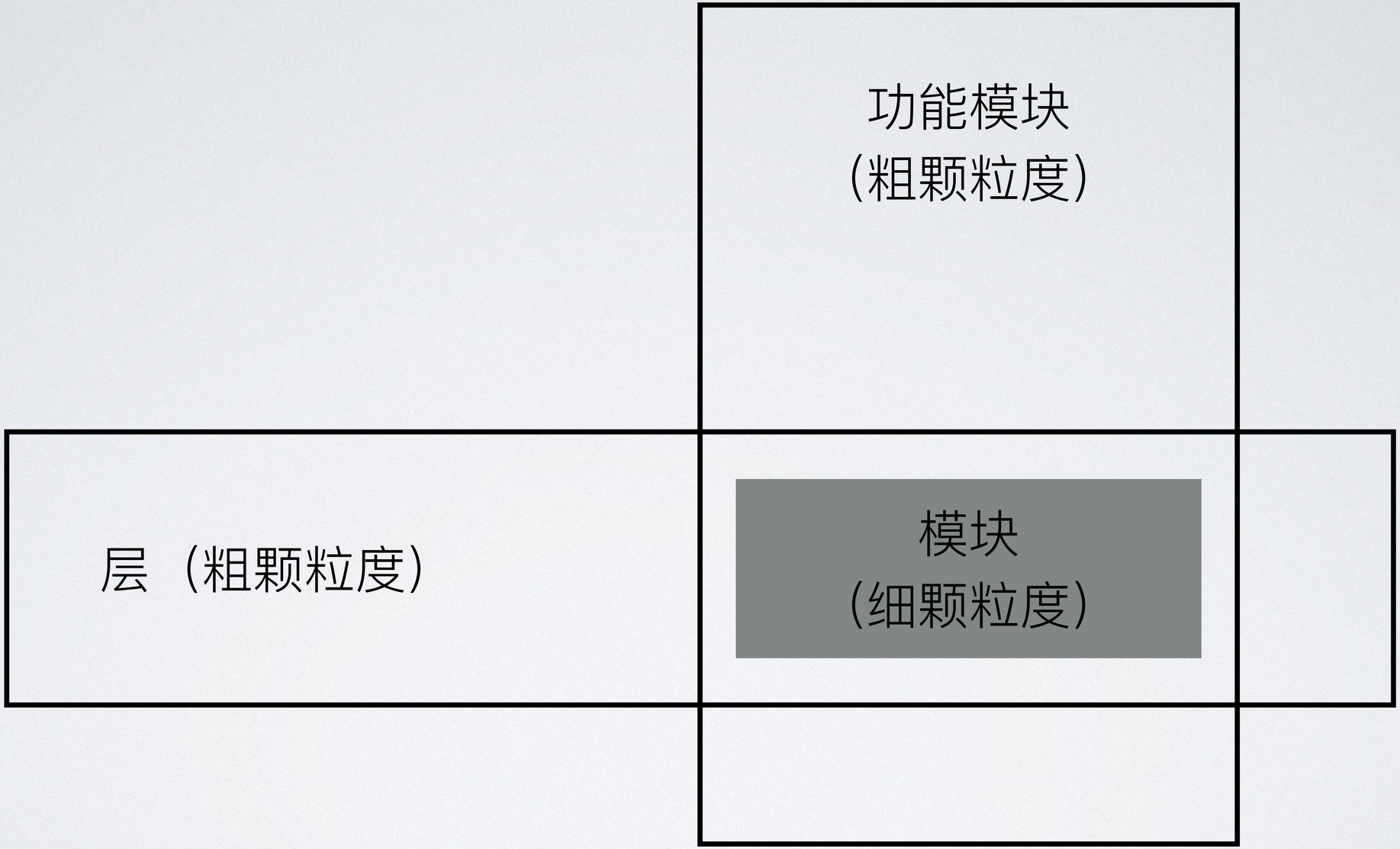
垂直切分（功能模块） vs 水平切分（层）

- 功能模块是粗颗粒度的，一般对应一个功能组，最大的用途是基于功能模块进行开发小组分工。

- 层也是粗颗粒度的，UI交互层封装人机交互、数据管理层封装DB，也是一种有价值且流行的关注度分离手段

- 但是，模块划分设计需要进一步到细颗粒度一级。

  一个细颗粒度模块，必然位于架构的某一层中
  一个细颗粒度模块，一般也都会属于某个更大颗粒度的功能模块内。

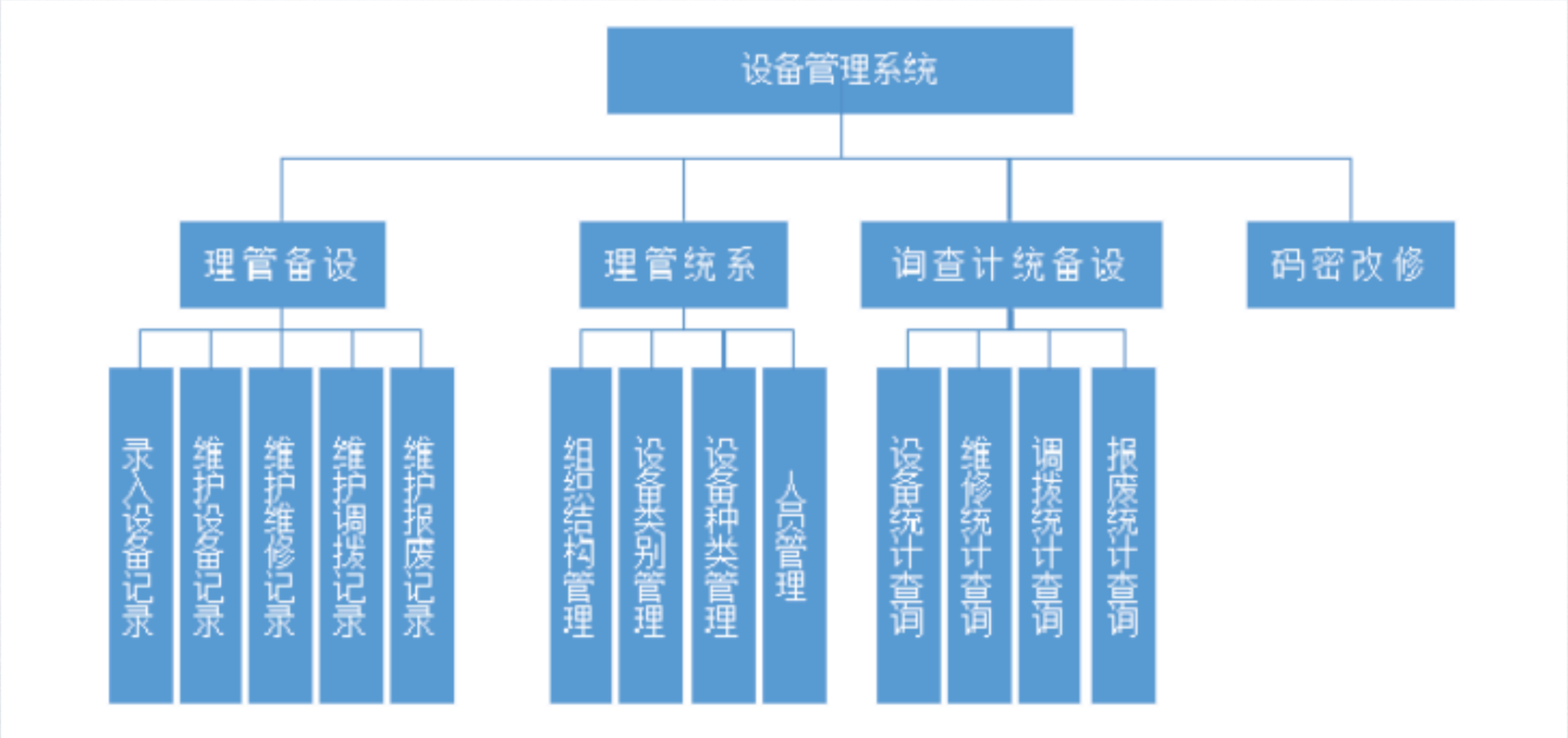

功能模块
（粗颗粒度）
层（粗颗粒度）
模块
（细颗粒度）

封装驱动设计方法

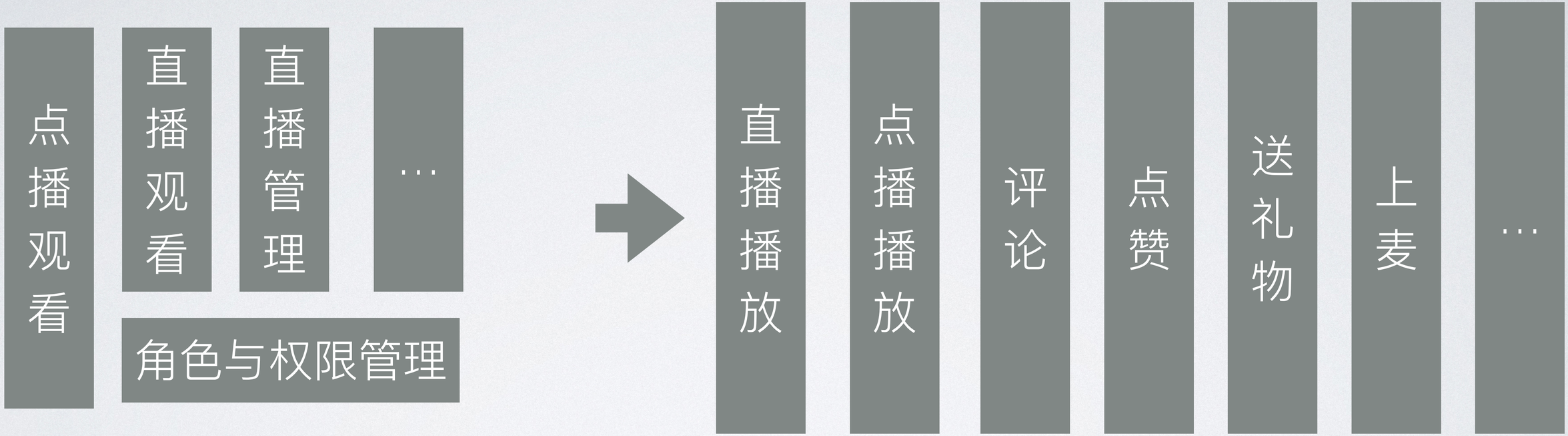
- 研究需求（粗颗粒度划分模块）
- 粗颗粒度分层
- 细颗粒度划分模块
- 用例驱动的模块划分结构评审、优化。

粗颗粒度划分模块（借助功能树）

功能树：一种功能表达结构，它将“功能大类”、“功能组”和“功能项”的隶属关系以“树”的形式呈现出来，非常直观。



核心原理：从“功能组”到“功能模块”



模块结构分解图

功能组

功能大类

音视频

互动

功能组

直播播放

上麦

点播播放

评论

点赞

送礼物

粗颗粒度分层

- 分层思想：封装，具体而言，是“封装外部交互”，明确每一层次职责，只关注领域本身的逻辑，

常用分层架构

- 基础层： 负责一些基础的服务，图片加载、网络请求等。
- 公共业务层： 负责提供公有的业务，比如，登录组件。
- 业务层： 具体的业务组件，比如，简历模块。

业务层	业务UI（职位列表等）
通用业务层	UI组件（视频宫格、消息列表等）
基础层	基础能力（网络、上麦、IM消息发送）

业务层

业务UI（职位列表等）

通用业务层

UI组件（视频宫格，消息列表等）

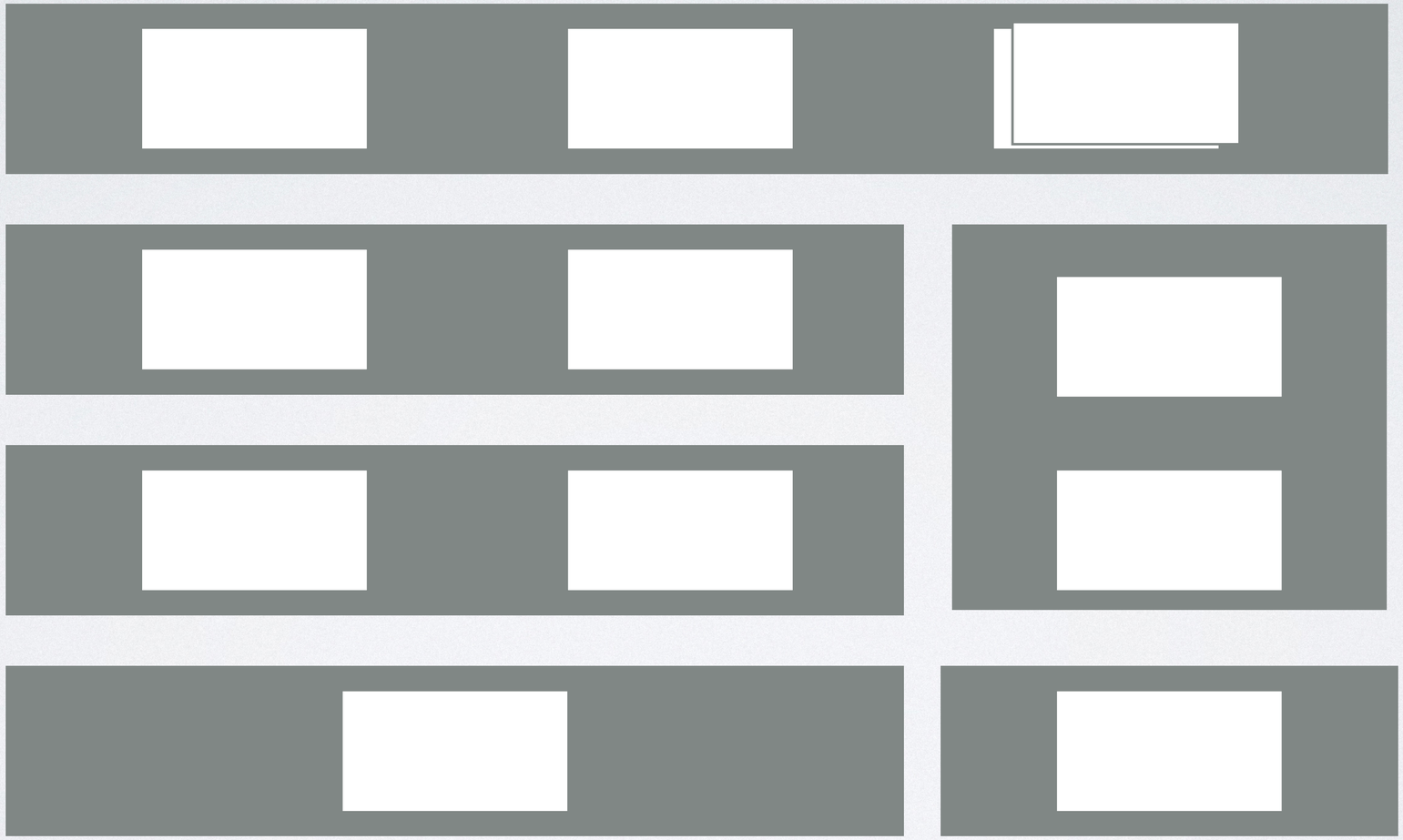
基础层

基础能力（开播，上麦，IM消息发

音视频

互动

细颗粒度分模块



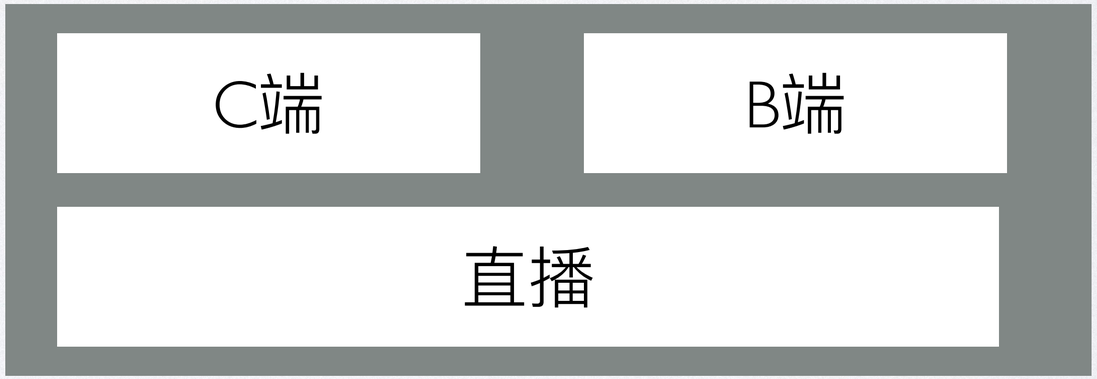
细粒度模块划分技能项

- 分层细化
- 分区
- 通用模块的分离
- 通用机制的框架化

分层细化

- 在粗粒度层内进一步“封装”职责形成更细致的层次结构，例如UI层内，View和Controller层的分离

业务层



分区

- 参考粗颗粒度划分技巧（从功能组到功能模块），将“层”内支持不同功能组的职责分别“封装”到“细颗粒度模块”中。

通用业务层 | UI组件（视频宫格，消息列表等）

通用业务层 | 视频画面组件 | 聊天消息组件 | 头部信息组件 | 底部操作组件

通用模块的分离

- 高层模块应有较高的扇出，低层模块特别是低层模块应有较高的扇入。
- 扇入越大，表示该模块被更多的上级模块共享。
- 运用参数化，抽象等设计手段，提炼通用模块，供其他多个模块调用，这就避免了程序的重复。

基础层

基础能力（开播，上麦，IM消息发送）

基础层

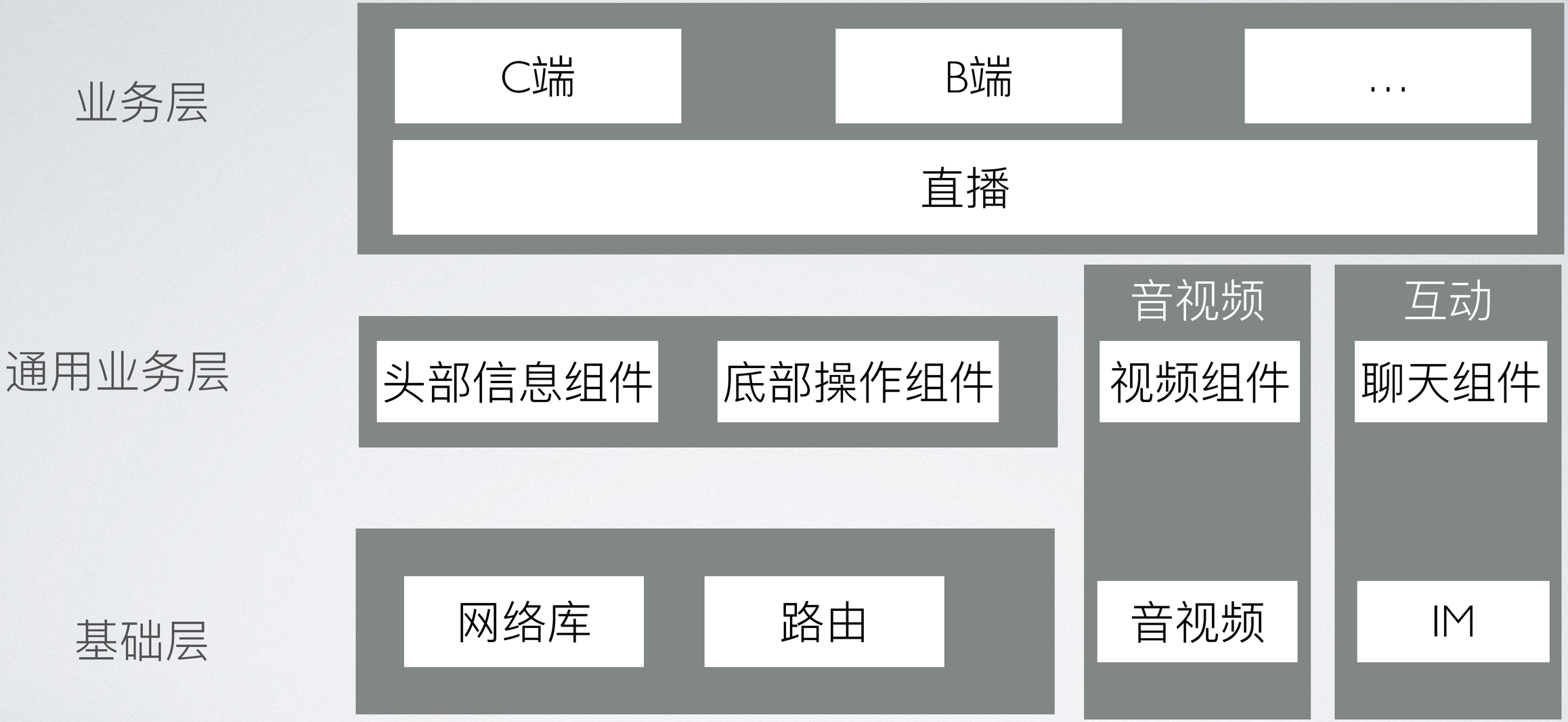
网络库

路由

…

通用机制的框架化

- 软件系统中的机制，是指预先定义好的、能够完成预期目标的、基于抽象角色的写作方式。机制不仅包含了协作关系，同时也包含了协作流程
- 没有提取机制的情况下，机制是一种隐式的重复代码。
- 框架是可以通过某种回调机制进行扩展的软件系统或子系统的半成品。
- 将容易变化的部分封装成扩展点，并辅以回调机制将他们纳入框架的控制范围内，从而在兼顾定制开销的同时使被重用的设计成果最多。


业务层
C端
B端
…
直播
通用业务层
头部信息组件
底部操作组件
音视频
视频组件
互动
聊天组件
基础层
网络库
路由
音视频
IM

评审、优化（用例驱动）

- 从用例到类，再到模块-实现用例需要哪些类，这些类需要归到哪些模块

分治思维

分治即分而治之。可以理解为将一个复杂的问题划分为多个子问题，分别进行处理，最后达到解决这个问题的目标。

在算法中，分治是一个非常流行的解决复杂问题的工具，它将一个问题划分为多个子问题，逐一处理，最后合并到一起，组成整个问题的解决方案。

如何掌握分治思维呢?

面对一个问题时，我们应该先对问题进行深刻的分析，确定问题是否能直接解决，如果不能直接解决，就考虑将问题分解，思考问题的组成元素，尝试将组成元素分离，确认某个元素是否已经有解决方案可以使用，逐一对各个元素进行分析。

迭代思维

没有任何一款优秀的架构可以一蹴而就，在架构创建完成后，持续收集用户反馈并进行迭代改进是必不可少的环节，这就是迭代思维，亦是演化思维和进化思维。

随着时间的推移和业务的变化，架构会被逐步侵蚀，软件熵会不断增加，迭代思维可以帮助架构降低熵增。

掌握迭代思维，能使架构师意识到并不是每一次混乱都需要继续容忍下去，推进迭代和重构，改善既有架构是一个架构师义不容辞的责任。

三、架构设计模式原则（SOLID）

- 单一职责原则（SRP）
- 开放封闭原则（OCP）
- 里氏替换原则（LSP）
- 依赖倒置原则（DIP）
- 接口分离原则（ISP）

四、架构设计步骤

- 认识系统：认识系统的数据模型、业务组成、模块组成等。
- 设计系统：选择合适的架构模式，对架构模式调研分析，确定系统架构设计方案。
- 创建系统：选择系统的实现环境，对系统进行部署，进入开发/改造阶段。
- 收集反馈：系统创建完成，持续调研，收集反馈，对系统做下一步优化规划

软件熵

熵的概念最早起源于物理学，用于度量一个热力学系统的无序程度。热力学第二定律，又称“熵增定律”，表明了在自然过程中，一个孤立的系统总是从最初的集中、有序的排列状态，趋向于分散、混乱和无序；当熵达到最大时，系统就会处于一种静寂状态。

通俗的讲：系统的熵增过程，就是由原始到死亡的过程。“熵”是“活跃”的反义词，代表负能量。

非生命，比如物质总是向着熵增演化，屋子不收拾会变乱，手机会越来越卡，耳机线会凌乱，热水会慢慢变凉，太阳会不断燃烧衰变……直到宇宙的尽头——热寂。

- 能量总和=有效能量+无效能量

在软件开发、维护过程中。软件的生命力总是从最初的理想状态，逐步趋向于复杂、混乱和无序状态发展，直到软件不可维护而被迫下线或重构。这种损坏软件质量的因素的逐步增长，叫做软件的熵增现象。